

Guión de Práctica 5: Memoria Dinámica II

Metodología de la Programación
Grado en Ingeniería Informática

Curso 2025/2026

1 Parte 1. Tabla Hash con resolución de colisiones mediante hashing abierto

1.1 Objetivos

Con este ejercicio se pretende que el alumno sea capaz de:

- Implementar un **Tipo de Dato Abstracto (TDA)** en C++.
- Comprender el funcionamiento de una **tabla hash**.
- Gestionar colisiones mediante **listas enlazadas**.
- Utilizar correctamente **memoria dinámica** (`new` y `delete`).
- Trabajar con **punteros y estructuras dinámicas**.

1.2 Descripción del problema

Se desea implementar una estructura de datos denominada **tabla hash** para almacenar números enteros.

La tabla tendrá un tamaño fijo y gestionará las colisiones mediante **encadenamiento**, es decir, cada posición del array contendrá una **lista enlazada simple**.

1.3 Especificación

1.3.1 Estructura interna

- La tabla será un array dinámico.
- Cada posición del array apuntará al inicio de una lista enlazada.
- Cada nodo de la lista contendrá:
 - Un valor entero.
 - Un puntero al siguiente nodo.

```

struct Nodo {
    int dato;
    Nodo* siguiente;
};

class TablaHash {
private:
    Nodo** tabla;    // array de listas (punteros a Nodo)
    int capacidad;

    int hash(int clave);

public:
    TablaHash(int capacidad);
    ~TablaHash();

    void insertar(int clave);
    bool buscar(int clave);
    bool eliminar(int clave);
    void mostrar();
};

```

1.3.2 Función hash

Se utilizará la siguiente función hash:

$$h(k) = k \% \text{capacidad}$$

donde k es la clave a insertar y `capacidad` el tamaño de la tabla.

1.3.3 Funcionalidades a implementar

Se debe implementar una clase `TablaHash` con los siguientes métodos:

Constructor

```
TablaHash(int capacidad);
```

- Reserva memoria dinámica para la tabla.
- Inicializa todas las posiciones a `nullptr`.

Destructor

```
~TablaHash();
```

- Libera toda la memoria utilizada:
 - Todos los nodos de las listas.
 - El array de la tabla.

Insertar

```
void insertar(int clave);
```

- Inserta el elemento en la tabla, en la primera posición de la lista correspondiente.
- Se permite insertar elementos duplicados.

Buscar

```
bool buscar(int clave);
```

- Devuelve `true` si el elemento está en la tabla.
- Devuelve `false` en caso contrario.

Eliminar

```
bool eliminar(int clave);
```

- Elimina la primera ocurrencia del elemento.
- Devuelve:
 - `true` si se elimina correctamente.
 - `false` si no existe.

Mostrar

```
void mostrar();
```

Formato orientativo:

```
0: 15 -> 25 -> NULL
1: NULL
2: 32 -> NULL
```

Programa de prueba

```
#include <iostream>
#include "HashTable.h"

using namespace std;

void menu() {
    cout << "\n===== TABLA HASH =====\n";
    cout << "1. Insertar elemento\n";
    cout << "2. Buscar elemento\n";
```

```

    cout << "3. Eliminar elemento\n";
    cout << "4. Mostrar tabla\n";
    cout << "0. Salir\n";
    cout << "Selecciona una opcion: ";
}

int main() {
    int capacidad;
    cout << "Introduce el tamaño de la tabla: ";
    cin >> capacidad;

    HashTable ht(capacidad);

    int opcion;
    int valor;

    do {
        menu();
        cin >> opcion;

        switch (opcion) {

            case 1:
                cout << "Introduce el valor a insertar: ";
                cin >> valor;
                ht.insertar(valor);
                cout << "Elemento insertado.\n";
                break;

            case 2:
                cout << "Introduce el valor a buscar: ";
                cin >> valor;
                if (ht.buscar(valor))
                    cout << "Elemento encontrado.\n";
                else
                    cout << "Elemento NO encontrado.\n";
                break;

            case 3:
                cout << "Introduce el valor a eliminar: ";
                cin >> valor;
                if (ht.eliminar(valor))
                    cout << "Elemento eliminado.\n";
                else
                    cout << "Elemento no existe.\n";
                break;

            case 4:
                cout << "\nContenido de la tabla:\n";

```

```

        ht.mostrar();
        break;

    case 0:
        cout << "Saliendo...\n";
        break;

    default:
        cout << "Opcion invalida.\n";
    }

    while (opcion != 0);

    return 0;
}

```

2 Parte 2. Juego de los barquitos

Se desea resolver el problema del juego de los barquitos. Para ello, se propone diseñar una clase `Barquitos` que contiene el tablero de un jugador. La clase debe incluir una matriz de enteros para codificar el estado del tablero. Dicho estado se representa con una estructura de dos dimensiones de tamaño variable, reservada con memoria dinámica.

	1	2	3	4	5	6	7	8	9	10	11
A											
B		.					.				
C			.	.	.	.	×	×	×	.	
D				×			.				
E							×				
F							.				
G											
H		.			.						
I		.			.						
J			.							×	
K							.	.			
L							×	×	.		

Codificación:

Valor negativo: se ha disparado en la casilla
 Valor 9: agua
 Valor entero i: parte de un barco de i casillas

	1	2	3	4	5	6	7	8	9	10	11
A	9	9	9	9	9	9	9	9	9	9	9
B	9	-9	1	9	9	9	-9	9	9	9	9
C	9	9	-9	-9	-9	-9	-3	-3	-3	-9	9
D	9	9	9	-4	9	9	-9	9	9	9	9
E	9	1	9	4	9	9	-1	9	9	9	9
F	9	9	9	4	9	9	-9	9	9	2	2
G	9	9	9	4	9	9	9	9	9	9	9
H	9	-9	9	9	-9	9	3	9	9	9	9
I	9	-9	9	9	-9	9	3	9	9	9	9
J	9	9	-9	9	9	9	3	9	9	-1	9
K	9	2	2	9	9	9	9	-9	-9	9	9
L	9	9	9	9	9	9	9	-2	-2	-9	9

Ejemplos:

B3: valor 1. Barco de 1 casilla
 B2: valor -9. Agua donde se ha disparado
 D4: valor -4. Barco de 4 alcanzado
 C8: valor -3. Barco de 3 alcanzado

2.1 Constructor

El constructor recibe como parámetros las dimensiones del tablero (número de filas y columnas) y construye un tablero en memoria dinámica en el que todas las posiciones contienen agua.

2.2 Colocar barco

Implemente un método que reciba la posición de un barco y determine si puede ser colocado en el tablero. En caso de ser posible, lo insertará en el tablero. La posición del barco vendrá determinada por:

- **Casilla de inicio:** Representada mediante una letra de la A a la Z para indicar la fila y un entero para indicar la columna (a partir del 1).
- **Tamaño del barco:** Un entero positivo mayor que 0 y menor que 9.
- **Dirección del barco:** 'H' o 'V', indicando que el barco se coloca en forma horizontal-derecha o vertical-abajo, respectivamente.

El barco puede ser colocado en el tablero siempre que todas las casillas a ocupar estén libres (agua) y no haya ningún otro barco en las casillas contiguas.

2.3 Disparar

Implemente un método que dispare a una posición del tablero. El método debe devolver:

- **1: Agua.** La posición disparada contenía agua y se marca como disparada.
- **2: Tocado.** La posición disparada contenía un barco y se marca como tocado.
- **3: Hundido.** Se ha disparado a la última casilla de un barco, quedando todas sus casillas tocadas.

3 Programa principal

```
#include <iostream>
#include <cstdlib>
using namespace std;

int main(int argc, char* argv[]) {

    if (argc != 3) {
        cout << "Uso: " << argv[0] << " <filas> <columnas>" << endl;
        return 1;
    }

    int filas = atoi(argv[1]);
    int columnas = atoi(argv[2]);

    Barquitos juego(filas, columnas);
```

```

cout << "=== TABLERO INICIAL ===" << endl;
juego.mostrar();

cout << "\n=== COLOCACION DE BARCOS ===" << endl;

// Barco base
cout << "Barco 1 (B2, tam 3, H): "
    << juego.colocarBarco('B', 2, 3, 'H') << endl;

// SOLAPAMIENTO
cout << "Barco 2 (B3, tam 2, H) [SOLAPA]: "
    << juego.colocarBarco('B', 3, 2, 'H') << endl;

// ROCE (adyacente)
cout << "Barco 3 (A2, tam 2, H) [ROZA]: "
    << juego.colocarBarco('A', 2, 2, 'H') << endl;

// válido
cout << "Barco 4 (E5, tam 2, V): "
    << juego.colocarBarco('E', 5, 2, 'V') << endl;

cout << "\n=== TABLERO ===" << endl;
juego.mostrar();

cout << "\n=== DISPAROS ===" << endl;

cout << "Disparo A1 (agua): "
    << juego.disparar('A', 1) << endl;

cout << "Disparo B2 (tocado): "
    << juego.disparar('B', 2) << endl;

cout << "Disparo B3 (tocado): "
    << juego.disparar('B', 3) << endl;

cout << "Disparo B4 (hundido): "
    << juego.disparar('B', 4) << endl;

cout << "\n=== TABLERO FINAL ===" << endl;
juego.mostrar();

return 0;
}

```